第五章 时序逻辑电路

时序逻辑电路（Sequential Logic Circuit）是数字系统中与组合逻辑电路并列的另一大类基本电路。组合逻辑电路的输出仅由当前输入决定，没有记忆能力；而时序逻辑电路的输出不仅取决于当前输入，还依赖于电路过去的历史状态——换言之，时序电路具有记忆功能。这一根本差异决定了时序逻辑电路比组合逻辑电路更复杂，分析与设计方法也更具挑战性。

时序逻辑电路在数字系统中的地位举足轻重。计算机处理器的程序计数器（PC）记录当前执行到哪条指令；寄存器文件（Register File）保存运算的中间结果；状态机（FSM）控制外设的工作流程；计数器统计脉冲个数并实现分频……这些功能无一不依赖时序逻辑电路来实现。可以说，没有时序逻辑就没有现代数字计算机。

本章在第四章触发器知识的基础上，系统介绍时序逻辑电路的基本概念与描述方法（5.1 节）、寄存器与移位寄存器（5.2 节）、各类计数器的工作原理与集成器件（5.3 节），以及同步时序逻辑电路的分析方法与设计方法（5.4、5.5 节）。本章内容是数字电子技术的核心与难点，需要认真理解状态方程、状态转换图等新工具，并通过大量例题建立扎实的分析与设计能力。

5.1 时序逻辑电路的基本概念

在深入学习各类具体时序电路之前，首先需要建立对时序逻辑电路整体结构特征和描述方法的清晰认识。

5.1.1 时序逻辑电路的结构特点

时序逻辑电路由两个部分组成：组合逻辑电路和存储电路（触发器组）。两部分的功能分工如下：组合逻辑电路根据外部输入信号 X(t) 和存储电路的现态 Q(n) 计算出两类结果——其一是电路的外部输出 Z(t)，其二是提供给触发器的激励信号（驱动信号）D(n)；存储电路则由若干触发器组成，在时钟沿到来时，根据激励信号将当前状态更新为次态 Q(n+1)，并将次态反馈给组合逻辑，形成闭环系统。

时序逻辑电路的基本结构框图如图 5-1 所示：

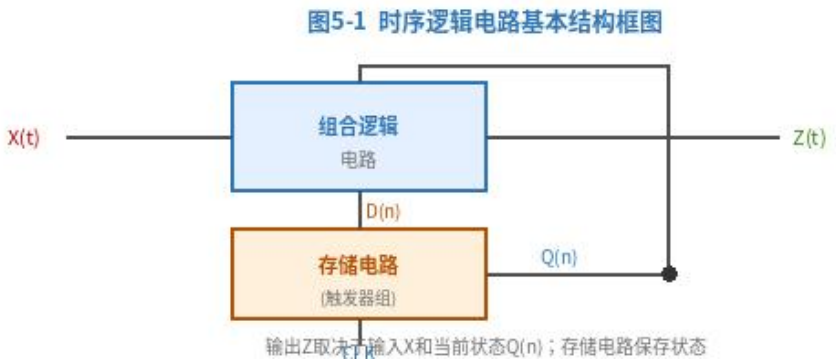


图 5-1 时序逻辑电路基本结构框图

从图 5-1 可以清晰看出时序电路的两个关键特征：第一，存在从存储电路（触发器）输出到组合逻辑输入的反馈路径，这是时序电路有别于组合电路的结构标志；第二，信号处理与存储分离，触发器在时钟统一控制下同步更新状态，赋予电路规律可控的时序行为。

用数学方程描述时序逻辑电路的行为，需要三组方程联立：

- 输出方程：描述外部输出 Z 与当前输入 X 和现态 Q(n) 的关系，Z = F[X(t), Q(n)]。
- 激励方程（驱动方程）：描述每个触发器的输入端信号（如 D 端）与 X 和 Q(n) 的关系，D = G[X(t), Q(n)]。
- 状态方程：将激励方程代入触发器特征方程，得到次态 Q(n+1) 与 X 和现态 Q(n) 的关系，Q(n+1) = H[X(t), Q(n)]。

这三组方程共同完整地描述了时序逻辑电路的全部逻辑行为，是时序电路分析与设计的数学基础。

5.1.2 时序电路的分类

时序逻辑电路可从多个角度进行分类：

1. 按时钟方式分：同步时序电路与异步时序电路

同步时序电路中，所有触发器共用同一个时钟信号 CLK，在每个时钟有效沿到来时，所有需要改变状态的触发器同步翻转，整个电路的状态只在时钟沿时刻发生变化。同步时序电路的分析和设计方法系统规范，时序行为可预测，是现代数字系统的主流设计模式。

异步时序电路中，不同触发器的时钟可能不同（例如，一个触发器的时钟由另一个触发器的输出驱动），各触发器的翻转不同步，存在先后顺序。异步电路的优点是不需要全局时钟，在某些低功耗场合有优势；缺点是分析复杂、容易产生竞争冒险，设计难度较大。

2. 按输出特性分：Moore 型与 Mealy 型

Moore 型时序电路的输出 Z 仅取决于电路的当前状态 Q(n)，与当前输入 X 无关，即 Z = F[Q(n)]。输出只在状态转换时改变，不会随输入的瞬时变化而波动，抗噪声能力较强，多用于对输出稳定性要求较高的场合。

Mealy 型时序电路的输出 Z 不仅取决于当前状态，还取决于当前输入，即 Z = F[X(t), Q(n)]。优点是状态数通常少于等效的 Moore 型电路（用更少的触发器实现相同功能），缺点是输出可能随输入的毛刺而产生毛刺，需要注意时序配合。

5.1.3 时序电路的描述方法

时序逻辑电路的逻辑功能有多种等价的描述方式，各有侧重：

状态方程组：用布尔代数方程组描述电路的状态转移和输出，是最精确的数学描述，便于计算机辅助分析。

状态转换表（次态表）：以表格形式列出所有现态和输入的组合，以及对应的次态和输出，直观完整，是设计与分析的核心工具。

状态转换图：以有向图形式描述状态机，状态为节点，转换条件和输出标注在有向边上。直观可视，便于理解状态机的工作流程和状态数量。

时序波形图：以时间轴展示各信号的波形变化，直观展示电路的动态工作过程，是验证设计正确性和解释波形关系的重要手段。

这四种描述方式之间可以相互转换，灵活运用是分析与设计时序电路的关键能力。在后续的分析和设计中，将根据具体问题的需要选择最合适的描述方式。

5.2 寄存器与移位寄存器

寄存器（Register）是存储若干位二进制数据的时序逻辑电路，是数字系统中最基础的存储单元之一。在计算机处理器中，通用寄存器组、指令寄存器、程序计数器等都是寄存器的具体应用。

5.2.1 基本寄存器

基本寄存器（也称并行寄存器）由 n 个 D 触发器和公共时钟组成，能够在时钟有效沿同时并行存储 n 位二进制数据，如图 5-2 所示。

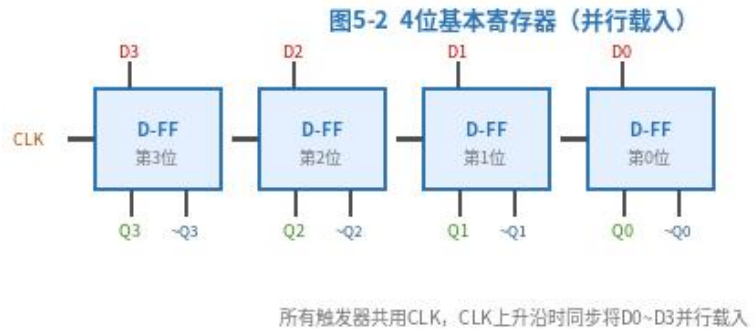


图 5-2 4 位基本寄存器（并行载入）

工作原理：在 CLK 上升沿到来时，D 端（D0~D3）的输入数据被同时并行地锁存到各触发器的 Q 端（Q0~Q3），完成一次数据写入；在两次时钟沿之间，输出 Q0~Q3 保持不变，实现数据的稳定存储，直到下一个时钟沿到来。

基本寄存器的功能特性：

- 数据并行输入，并行输出（PIPO，Parallel In Parallel Out）。
- 只能整体替换所有位的数据，不能对个别位单独操作。
- 每次存储操作需要一个时钟周期（一个时钟沿）完成。
- 配合三态缓冲器（如 74HC374），可以连接到总线上，实现总线上的数据寄存和总线驱动控制。

常用集成寄存器芯片：74HC374（8 位上升沿 D 触发器，三态输出，～OE 使能）是总线接口的标准选择；74HC273（8 位 D 触发器，~CLR 异步清零，无三态输出）适合内部数据暂存；74HC175（4 位 D 触发器，同步清零）是通用的 4 位寄存单元。

5.2.2 移位寄存器

移位寄存器（Shift Register）不仅能存储数据，还能在时钟脉冲的控制下，将存储的数据逐位向左或向右移动，实现数据的串行传输和代码格式转换。

1. 单向移位寄存器

4 位右移移位寄存器的电路结构如图 5-3 所示，由 4 个 D 触发器串联而成，每个触发器的 Q 端连接下一个触发器的 D 端，首端触发器的 D 端接串行输入 Sin。

图5-3 4位右移移位寄存器（串行输入）

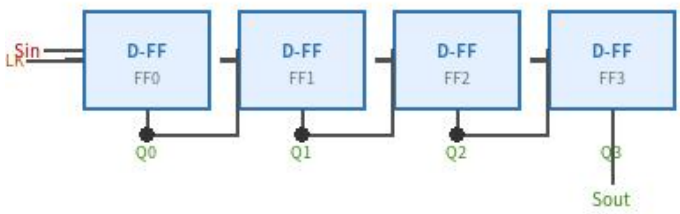


图 5-3 4 位右移移位寄存器

工作原理：每来一个时钟上升沿，FF0 从 Sin 接收新的串行输入位，同时各触发器将自身的 Q 值（现态）传给下一个触发器，实现数据整体右移一位。经过 4 个时钟周期后，4 位串行输入数据依次进入寄存器，可以从 Q0~Q3 并行读出，完成串行到并行（SIPO）的格式转换。

移位寄存器的状态转换表（以初始全 0，串行输入 1011 为例）：

CLK 沿次数	Sin 输入	Q0	Q1	Q2	Q3	说明
初始	—	0	0	0	0	全零初始状态
第 1 沿	1	1	0	0	0	1 进入 Q0
第 2 沿	0	0	1	0	0	右移，0 进入 Q0
第 3 沿	1	1	0	1	0	右移，1 进入 Q0

第 4 沿	1	1	1	0	1	右移，1 进入 Q0（最终状态）

第 4 个时钟沿后，Q3Q2Q1Q0 = 1011，与串行输入顺序（先入 Q0，后移到 Q3）一致，完成 4 位串行数据的并行读出。

2. 双向移位寄存器——74HC194

74HC194 是一款功能完整的 4 位双向移位寄存器，其逻辑符号如图 5-4 所示。

图5-4 4位双向移位寄存器74HC194逻辑符号

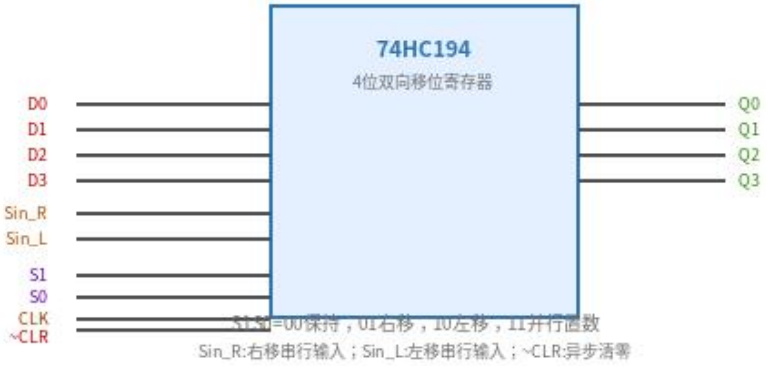


图 5-4 4 位双向移位寄存器 74HC194 逻辑符号

74HC194 的功能由两位工作方式控制端 S1、S0 共同决定：

S1	S0	工作方式	功能描述
0	0	保持	Q 保持不变，时钟不影响状态
0	1	右移	数据从 Sin_R 端串行移入，Q0→Q1→Q2→Q3 方向移位
1	0	左移	数据从 Sin_L 端串行移入，Q3→Q2→Q1→Q0 方向移位
1	1	并行置数	在 CLK 上升沿将 D0~D3 并行载入 Q0~Q3

异步清零端 ~CLR 低有效，不受时钟控制，~CLR=0 时立即将 Q0~Q3 全部清零，与 S1S0 的设置无关。

74HC194 的典型应用场景：串并/并串转换（通信接口）、数据缓冲和格式变换、构成环形计数器和扭环计数器、以及多功能移位运算器（左移/右移实现乘以 2/除以 2）。

5.2.3 移位寄存器的典型应用

1. 串行-并行（SIPO）转换

将 n 个时钟周期的串行输入数据转换为 n 位并行输出。方法：将移位寄存器配置为右移模式（S1=0,S0=1），从 Sin_R 端逐位输入数据，经过 n 个时钟周期后，Q0~Q(n-1) 同时输出全部 n 位数据。这是 SPI（串行外设接口）、I2C 等串行通信协议的接收端核心电路。

2. 并行-串行（PISO）转换

将 n 位并行数据转换为串行输出。方法：先用并行置数功能（S1=1,S0=1）在一个时钟周期内将数据载入；然后切换到移位模式，连续 n 个时钟周期将数据逐位从 Q3 端串行输出。串行通信协议的发送端正是基于这一原理工作的。

3. 环形计数器

将移位寄存器末位的 Q 端反馈连接到 Sin 端，就构成了环形计数器（Ring Counter），如图 5-14 所示。4 位环形计数器有 4 个有效状态（1000、0100、0010、0001），4 个时钟周期一个循环，常用于产生顺序脉冲（节拍信号）。

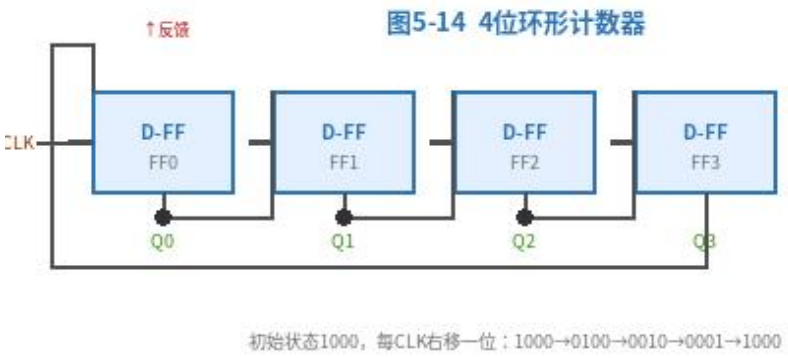


图 5-14 4 位环形计数器

环形计数器的特点：电路简单，自解码（每个状态下只有一个 Q 为 1，直接就是独热编码），无需额外译码电路；缺点是状态利用率低（4 个触发器只有 4 个有效状态，而不是最多的 16 个状态）。

4. 扭环形计数器（Johnson 计数器）

将移位寄存器末位的 Q' 端（Q 的反变量）反馈到 Sin 端，就构成了扭环形计数器（Twisted Ring Counter），也称 Johnson 计数器。n 位扭环形计数器有 2n 个有效状态（而环形计数器只有 n 个），状态利用率是环形计数器的 2 倍。4 位扭环形计数器的状态序列为：0000→1000→1100→1110→1111→0111→0011→0001→0000，共 8 个状态，是 4 位环形计数器（4 个状态）的 2 倍。扭环形计数器产生的序列相邻状态间只有 1 位变化，配合简单的译码电路可以产生无毛刺的序列脉冲。

5.3 计数器

计数器（Counter）是记忆输入脉冲个数、并在达到预定计数值时产生输出信号的时序逻辑电路。它是数字系统中使用频率最高的功能部件之一，广泛用于计数、定时、分频、程序控制、地址生成等场合。

5.3.1 计数器的分类

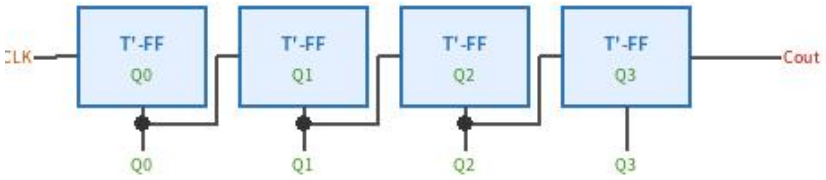
计数器的种类繁多，可以从不同角度进行分类：

分类维度	类别	特点说明
时钟方式	同步计数器	所有触发器共用 CLK，状态同步改变，速度快，设计规范
	异步计数器（行波计数器）	前级 Q 驱动后级 CLK，逐级异步翻转，结构简单但速度慢
计数方向	加法计数器	状态按 0,1,2,...,M-1,0,...循环递增
	减法计数器	状态按 M-1,...,1,0,M-1,...循环递减
	可逆计数器（加减计数器）	由控制信号决定加法还是减法方向
计数进制	二进制计数器	模 M=2^n，用 n 个触发器自然编码
	十进制计数器	模 M=10，8421BCD 码编码，需要修正逻辑
	任意 N 进制	M 可以是任意正整数，通过反馈法实现

5.3.2 4 位异步二进制加法计数器

异步二进制加法计数器是最简单的计数器结构，利用 T' 触发器（每个时钟沿翻转一次）的级联实现，如图 5-5 所示。

图5-5 4位异步二进制加法计数器（行波进位）



每个触发器恒T=1，Q的下降沿触发下一位翻转，各位异步工作

图 5-5 4 位异步二进制加法计数器

工作原理：每个触发器都配置为 T' 工作方式（JK 触发器令 J=K=1，或 D 触发器将 Q' 反馈到 D）。最低位 FF0 的时钟接系统时钟 CLK，其余各位的时钟接前一位的 Q 端。

计数规律（以二进制加法为例）：当 Q0 从 1 变为 0（下降沿）时，触发 FF1 翻转；当 Q1 从 1 变为 0 时，触发 FF2 翻转；以此类推，构成行波进位（Ripple Carry）结构。4 位计数器状态从 0000 经历 0001、0010.....1111 后回到 0000，循环计数，模数 M=16。

异步计数器的优点：结构极为简单，每位只需一个触发器和一根连线；缺点：进位以'行波'方式从低位向高位逐级传播，最高位的稳定需要等待 n 个触发器延迟，n 越大总延迟越长，导致最高工作频率受限。此外，由于各位不同步翻转，在状态转换过程中可能出现短暂的中间状态（竞争冒险），被高速采样电路误读。因此异步计数器适合速度要求不高的场合。

5.3.3 4 位同步二进制加法计数器

同步二进制计数器克服了异步计数器速度慢的缺点，所有触发器共用同一 CLK，在时钟沿时同步翻转，如图 5-6 所示。

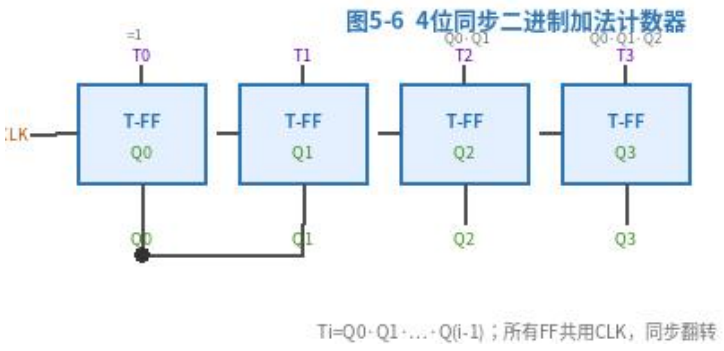


图 5-6 4 位同步二进制加法计数器

触发器翻转条件（T 触发器，T=1 时翻转）：

一般规律：第 i 位触发器的 T 输入为低 i 位输出的与：Ti = Q0·Q1·...·Q(i-1)。这来自于二进制加法的进位规则：只有当所有低位均为 1 时，本位才会在下一时钟沿产生进位（翻转）。

同步计数器的优点：所有触发器同时翻转（仅有一个触发器延迟），工作频率高，无中间状态问题；缺点：随位数增加，T 端的与门需要越来越多的输入，电路复杂度增加，高位 T 的计算延迟也会增加。实际集成同步计数器采用超前进位方案，将延迟控制在常数级别。

5.3.4 集成 4 位同步计数器 74HC161/163

74HC161 是一款功能强大的集成 4 位同步二进制加法计数器，其逻辑符号如图 5-7 所示。

图 5-7 4 位同步二进制计数器 74HC161 逻辑符号

74HC161 的引脚功能说明：

引脚	类型	功能说明
D0~D3	输入	4 位并行预置数输入端
CLK	输入	时钟上升沿触发
~CLR	输入	异步清零端（低有效），不受时钟控制，立即将 Q0~Q3 清零
~LOAD	输入	同步置数端（低有效），CLK 上升沿时将 D0~D3 并行载入 Q0~Q3
ENP	输入	计数使能端（高有效），ENP=1 且 ENT=1 时允许计数
ENT	输入	进位使能端（高有效），同时控制 RCO 输出
Q0~Q3	输出	4 位计数器输出，Q0 为最低位

RCO	输出	进位输出，Q0=Q1=Q2=Q3=1 且 ENT=1 时，RCO=1（宽度一个时钟周期）

74HC161 的功能优先级（从高到低）：~CLR（异步清零）> ~LOAD（同步置数）> ENP·ENT（计数）> 保持。即只要 ~CLR=0，无论其他信号如何，输出立即清零；~CLR=1 且 ~LOAD=0 时，在 CLK 上升沿置数；两者均无效且 ENP=ENT=1 时正常计数；否则保持不变。

74HC163 与 74HC161 的区别：161 的 ~CLR 是异步清零（立即生效），163 的 CLR 是同步清零（CLK 上升沿时才清零）。在需要消除毛刺的同步设计中，74HC163 更常用；在需要上电快速初始化的场合，74HC161 的异步清零更方便。

74HC163 的功能表（CLK 上升沿触发，按优先级排列）：

~CLR	~LOAD	ENP	ENT	Q(n+1)	功能
0	x	x	x	0000	同步清零（163），异步清零（161）
1	0	x	x	D3D2D1D0	同步置数，载入 D 端数据
1	1	1	1	Q+1	正常计数，模 16 循环
1	1	0	x	Q 不变	保持（ENP 无效）
1	1	x	0	Q 不变	保持（ENT 无效）

5.3.5 用集成计数器构成任意进制计数器

74HC161/163 是模 16（M=16）的二进制计数器，在实际工程中，往往需要模数不等于 16 的计数器（如模 10 的十进制计数器、模 12 的时钟计数器、模 60 的分/秒计数器等）。利用 74HC161/163 的控制端，可以通过两种方法实现任意模数 N 的计数器（N<16）。

方法一：置零法（反馈清零法）

利用 ~CLR 端（74HC161）或 CLR 端（74HC163）实现强制清零，跳过 M-N 个状态，仅保留 N 个状态（0 到 N-1）。

设计原理：当计数器状态到达 N 时（而不是等到 15），立即通过检测逻辑将 ~CLR 拉低，强制计数器清零回到 0000，从而实现模 N 计数。

以六进制（N=6）为例，如图 5-8 所示：当 Q2Q1Q0Q3 的状态达到 0110（十进制 6）时，Q1=1 且 Q2=1，用与非门检测这一条件，将输出连接到 ~CLR 端，触发清零。由于异步清零（74HC161）或状态 6 出现后立即消失（74HC163），状态 6 只是一个极短暂的过渡状态，实际有效状态为 0000~0101，共 6 个状态，实现了六进制计数。

图5-8 用置零法将74HC161改为六进制计数器

图 5-8 用置零法构成六进制计数器

置零法的要点：检测的状态是 N（而非 N-1），清零信号到达后状态立即从 N 跳回到 0；对于 74HC161（异步清零），状态 N 出现时间极短（约一个门延迟），是一个'稍纵即逝'的状态；74HC163 的同步清零则在下一个 CLK 沿才清零，因此状态 N 会完整持续一个时钟周期，也算作有效状态，需要检测状态 N-1 来触发下一周期的清零。

方法二：置数法（反馈置数法）

利用 ~LOAD 端实现强制置数到指定初始值，跳过 M-N 个状态。

设计原理：当计数器状态到达最大值（对于加法计数，就是 N-1）时，通过逻辑检测，在下一个 CLK 沿将计数器置回到指定的起始状态（通常是 0 或某个偏置值）。

以十进制（N=10）为例，如图 5-9 所示：检测 Q3=1 且 Q1=1 的条件（状态 10=1010，即十进制 10），将与非门输出连到 ~LOAD 端，同时将 D0~D3 置为 0000，则下一 CLK 沿到来时置数为 0000，实现 0000→0001→...→1001→0000 的十进制循环计数。

图5-9 用置数法将74HC161改为十进制计数器

图 5-9 用置数法构成十进制计数器（BCD 计数器）

置数法的要点：检测的是需要'跳回'的状态，或利用 RCO 进位输出（当计到 1111 时 RCO=1）来触发置数；起始值不必为 0，可以设置为任意初始值，灵活构造任意范围的计数器。例如，从 5 计数到 14（共 10 个状态），只需将 D0~D3 设置为 0101（5），在计数到 1111 时（或检测到 1110 后下一步）触发置数，即实现 5~14 的十进制计数。

5.3.6 计数器的级联扩展

当单片 74HC161（最大模 16）不能满足计数需求时，可以将多片级联以扩展计数范围。图 5-10 给出了两片 74HC161 同步级联的方案。

图5-10 两片74HC161级联构成8位计数器

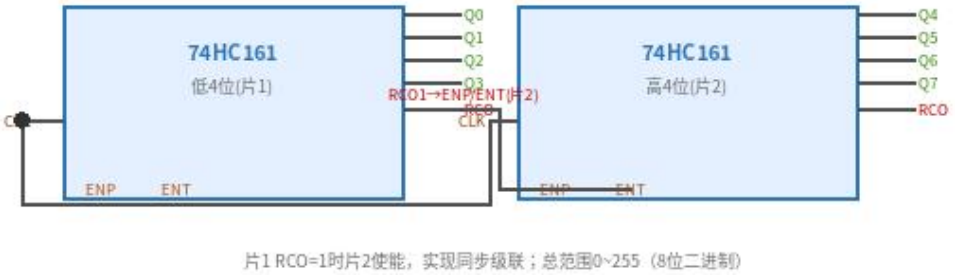


图 5-10 两片 74HC161 级联构成 8 位（模 256）计数器

级联原理：两片共用同一 CLK；低位片（片 1）的 ENP、ENT 均接高电平，始终处于计数状态；低位片的 RCO 输出连接到高位片（片 2）的 ENP 和 ENT 端。

工作过程：低位片从 0000 计到 1111（16 个状态）时，RCO=1，这使得在下一个 CLK 上升沿，高位片满足 ENP=ENT=1 的条件，状态加 1；同时低位片由于计数到最大值后自动回到 0000（自然溢出）。因此，8 位级联计数器实现了 0~255（十进制）的循环计数，模数 M = 16 × 16 = 256。

n 片 74HC161 级联后计数范围为 0 ~ 16^n - 1，可通过进一步添加置零/置数反馈逻辑构成任意模数计数器。例如，两片级联后用置零法构成模 100 的十进制两位 BCD 计数器，就是常见的秒计数器电路的基础。

5.4 时序逻辑电路的分析方法

对给定的时序逻辑电路，通过系统化的步骤推导其逻辑功能，这一过程称为时序电路的分析。与组合逻辑分析相比，时序电路分析引入了状态方程、状态转换表和状态转换图等新工具，步骤也更为严谨。

图5-11 同步时序电路分析步骤

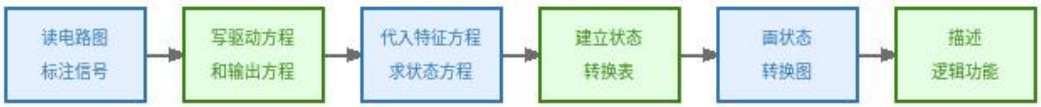


图 5-11 同步时序电路分析步骤

5.4.1 同步时序电路的分析步骤

读电路图，明确输入变量、输出变量和每个触发器的类型与编号，标注所有中间信号。

写驱动方程：根据组合逻辑部分的连接关系，写出每个触发器激励端（如 D 端、J/K 端）关于输入 X 和现态 Q(n) 的逻辑表达式。

写输出方程：根据输出端的连接关系，写出输出 Z 关于 X 和 Q(n) 的表达式（Moore 型输出只含 Q(n)）。

求状态方程（次态方程）：将驱动方程代入对应触发器的特征方程（D 触发器：Q(n+1)=D；JK 触发器：Q(n+1)=JQ'+K'Q），得到每个触发器的次态关于现态和输入的方程组。

建立状态转换表：将所有现态和输入的组合代入状态方程，计算次态和输出，填入表格。

画状态转换图：根据状态转换表，以圆圈表示状态，有向箭头表示转换，标注转换条件和输出，得到直观的状态图。

描述逻辑功能：对照状态转换图，分析状态序列的规律，用简洁语言描述电路实现的逻辑功能（如三位二进制加法计数器、序列检测器等）。

5.4.2 分析举例

【例 5-1】分析图 5-6 所示同步 4 位二进制加法计数器的工作原理

电路由 4 个 T 触发器（FF0~FF3）和组合逻辑构成，共用 CLK。

第一步，写驱动方程：

第二步，代入 T 触发器特征方程 Q(n+1) = T⊕Q(n)，得状态方程：

第三步，列状态转换表（16 行，此处列出前 5 行验证）：

Q3Q2Q1Q0（现态）	Q3+Q2+Q1+Q0+（次态）	说明
0000	0001	0+1=1
0001	0010	1+1=2
0010	0011	2+1=3
0011	0100	3+1=4
...	...	...
1111	0000	15+1=0（回零）

第四步，时序波形如图 5-13 所示（以 3 位计数器为例）：

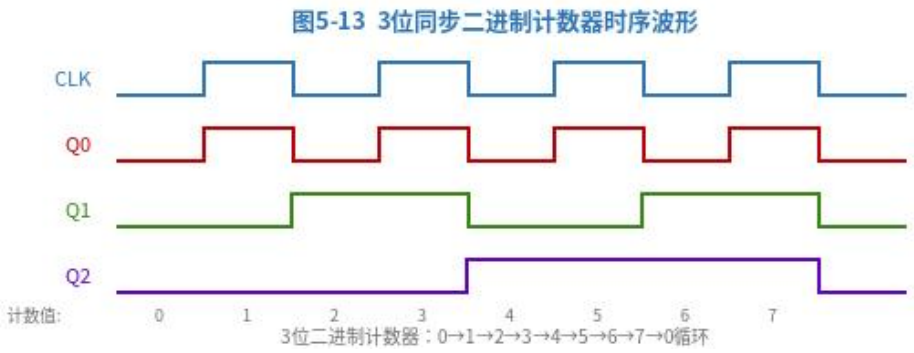


图 5-13 3 位同步二进制计数器时序波形

功能描述：4 位同步二进制加法计数器，从 0000 开始，每来一个时钟上升沿状态加 1，计到 1111（十进制 15）后再加 1 回到 0000，循环计数，模数 M=16（即 16 分频器）。进位输出（RCO 或 TC）在状态 1111 时输出高电平，可用于多片级联。

【例 5-2】分析三位同步时序电路的逻辑功能

设一个时序电路含 3 个 D 触发器（FF0、FF1、FF2，Q0 为最低位），驱动方程为：D0=Q2', D1=Q0, D2=Q1（无外部输入，Moore 型）；输出 Y=Q2。

写状态方程（D 触发器：Q(n+1)=D）：

列状态转换表（无输入变量，逐步递推）：

现态 Q2Q1Q0	次态 Q2+Q1+Q0+	输出 Y=Q2	说明
000	100	0	0→4
100	010	1	4→2
010	001	0	2→1
001	000	0	1→0
---	---	---	形成循环
101	100	1	非主循环状态
011	101	0	非主循环状态
110	011	1	非主循环状态
111	011	1	非主循环状态

功能描述：主循环 000→100→010→001→000 构成 4 状态循环序列，可检查自启动性——从非主循环状态出发，最终能否进入主循环。从上表可知，所有非主循环状态（101→100、011→101→100、110→011→101→100、111→011）最终都能进入主循环，因此电路具有自启动能力。输出 Y=Q2 在主循环中的序列为 0,1,0,0,...，产生周期脉冲，该电路是一个 4 分频电路（CLK÷4）。

5.5 时序逻辑电路的设计方法

时序电路设计是分析的逆过程：根据给定的逻辑功能要求，通过系统化步骤设计出能够实现该功能的时序电路。时序电路的设计比组合电路更复杂，核心是状态机的建立和化简，以及触发器类型的选择和驱动方程的推导。

图5-12 同步时序电路设计步骤

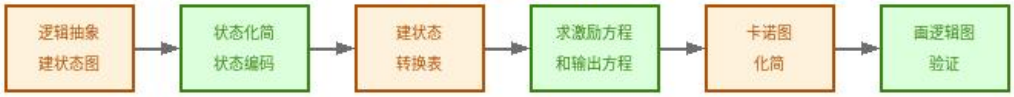


图 5-12 同步时序电路设计步骤

5.5.1 同步时序电路的设计步骤

逻辑抽象：仔细理解题意，确定输入变量、输出变量，以及需要'记忆'的信息（状态），画出原始状态转换图。这是设计中最具创造性的一步，直接决定后续设计的正确性。

状态化简：若原始状态图中存在等价状态（相同输入下产生相同输出和相同次态的两个状态），则合并等价状态，减少状态总数，从而减少触发器数量。

状态编码：为每个状态分配唯一的二进制编码。编码方式的选择（自然二进制、格雷码、独热码等）影响后续驱动方程的复杂度。原则是尽量使卡诺图化简效果好（相邻状态编码相差 1 位）。n 个状态需要 $\log_2 n$ 个触发器（向上取整）。

建立状态转换表：将状态转换图中的转换关系，结合状态编码，以表格形式列出所有现态、输入、次态和输出。

推导驱动方程和输出方程：根据状态转换表，对应所选触发器的激励表（四种触发器的激励表，参见第四章），求出每个触发器输入端的逻辑表达式，用卡诺图化简（充分利用无关项——不会出现的状态对应的输出全为无关项）。

画逻辑电路图：根据化简后的驱动方程和输出方程，选用适当的逻辑门和触发器，画出完整的电路图。

检查自启动性：验证所有未被使用的状态（无效状态）在任何输入条件下能否最终进入有效状态循环。若存在无法自启动的无效状态，需修改设计添加自启动逻辑。

5.5.2 设计举例

【例 5-3】设计一个同步三进制（模 3）加法计数器

要求：用 D 触发器实现，状态序列为 00→01→10→00（2 个触发器，3 个有效状态）。

建立状态转换表（Q1Q0 为现态，Q1+Q0+为次态）：

Q1	Q0	Q1+	Q0+	说明
0	0	0	1	0→1
0	1	1	0	1→2
1	0	0	0	2→0
1	1	×	×	无效状态（无关项）

用 D 触发器，驱动方程即为次态：D1 = Q1+，D0 = Q0+。用卡诺图（含无关项 m3）化简：

验证：初始 Q1Q0=00，D1=0·1=0，D0=1·1=1，次态 01（状态 1）；再次 Q1Q0=01，D1=1·1=1，D0=0·1=0，次态 10（状态 2）；再次 Q1Q0=10，D1=0·0=0，D0=0·0=0... 等等，D0=~Q0·~Q1=0·1=0，次态 00，正确。

自启动检查：无效状态 Q1Q0=11，代入 D1=Q0·~Q1=1·0=0，D0=~Q0·~Q1=0·0=0，次态=00，进入有效循环。电路具有自启动能力。

【例 5-4】设计一个序列检测器，检测输入串行序列中是否出现'101'

功能要求：输入为串行信号 X（一次一位），当最近三位输入恰好为 1、0、1 时，输出 Z=1，否则 Z=0。这是 Mealy 型状态机的典型设计。

状态定义（根据已匹配的前缀长度）：S0-未匹配（初始状态）；S1-已匹配'1'；S2-已匹配'10'；S3-已匹配'101'（输出 1）。状态转换图如图 5-15 所示：

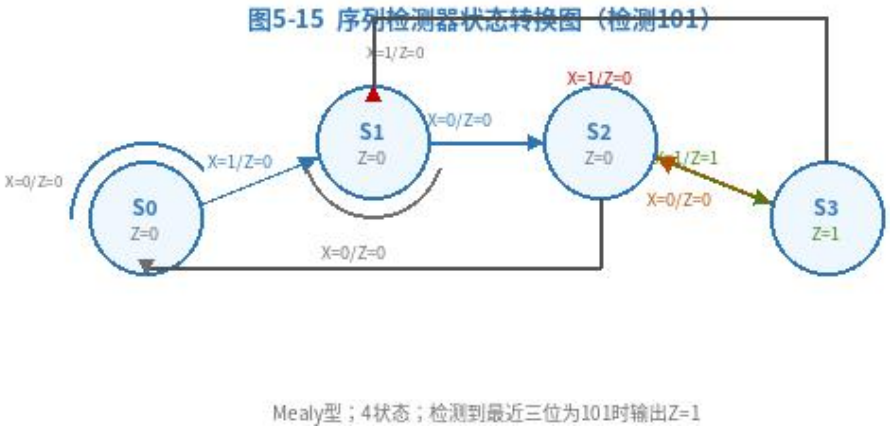


图 5-15 序列检测器状态转换图（检测'101'）

从状态图得到状态转换表（Mealy 型，输出标在转换条件后）：

现态	X=0 时次态/输出	X=1 时次态/输出
S0（00）	S0（00）/ Z=0	S1（01）/ Z=0
S1（01）	S2（10）/ Z=0	S1（01）/ Z=0
S2（10）	S0（00）/ Z=0	S3（11）/ Z=1
S3（11）	S2（10）/ Z=0	S1（01）/ Z=0

编码：S0=00，S1=01，S2=10，S3=11（4 个状态需要 2 位，Q1Q0 编码）。

推导 D 触发器驱动方程（用卡诺图化简，此处直接给出结果）：

电路实现：两个 D 触发器加上组合逻辑（若干与门和或门/非门）即可。序列检测器是状态机设计最经典的应用，在通信协议（帧同步、位同步）、密码识别、波形检测等场合有广泛用途。

【例 5-5】设计一个能自动启动的模 6 计数器

要求：3 位计数器，计数序列为 000→001→010→011→100→101→000，模 6，若意外进入无效状态 110 或 111，能自动回到有效循环，用 JK 触发器实现。

设计步骤：3 个有效循环状态外加状态 110（次态→000）和 111（次态→000）作为无效态处理。用卡诺图化简 JK 触发器的激励（利用无效状态作为无关项）：

验证自启动：状态 110，代入以上方程计算次态=001（进入有效循环）；状态 111，次态=000（进入有效循环），设计正确。

5.6 顺序脉冲发生器与序列信号发生器

在数字控制系统中，经常需要产生按一定时序关系发出的节拍控制信号（顺序脉冲）或特定的周期性二进制序列（序列信号）。本节介绍两类典型的时序信号发生电路。

5.6.1 顺序脉冲发生器

顺序脉冲发生器（Sequential Pulse Generator）能够在每个时钟周期依次产生一个有效脉冲，各脉冲在时间上不重叠，按固定顺序循环出现。这是数字系统节拍控制（如 CPU 的取指-译码-执行流水线控制）的基础。

实现方法：用 n 位环形计数器可以产生 n 个顺序脉冲，每个触发器的 Q 端输出对应一路脉冲；或者用 n 位扭环形计数器配合译码器，产生 2n 路脉冲。

以 4 位环形计数器（初始状态 1000）为例，4 个时钟周期内依次产生 Q0=1、Q1=1、Q2=1、Q3=1 各一个时钟周期宽度的脉冲，循环往复。这 4 路脉冲可直接驱动 4 个不同的控制信号，实现严格的时序控制，无需额外的译码电路。

5.6.2 序列信号发生器

序列信号发生器（Sequence Generator）产生按预设序列循环变化的周期性二进制序列。例如，需要产生序列 10110100 循环重复输出的电路，就是一个周期为 8 的序列信号发生器。

实现方法一——移位寄存器加反馈：n 位移位寄存器的末位输出通过组合逻辑反馈到串行输入端，反馈逻辑的设计决定了产生的序列内容。特殊地，当反馈函数取某些特殊形式时，能产生周期为 2n-1 的最长序列（m 序列），具有优良的伪随机特性，在通信和测量中有重要应用。

实现方法二——计数器加组合逻辑：用计数器产生依次变化的地址，组合逻辑（如 ROM 或数据选择器）根据地址输出预设的序列值。这种方法灵活性最强，可以产生任意预设序列，只需更改 ROM 或接线内容即可改变序列，适合工程应用。

5.7 时序电路的 Verilog HDL 描述

在现代数字系统设计中，时序逻辑电路通过硬件描述语言实现，综合工具自动将行为描述映射为最优的触发器和组合逻辑网表。本节给出本章各主要电路的 Verilog 参考实现。

5.7.1 基本寄存器与移位寄存器

```
// 8 位并行寄存器（上升沿，带异步复位）
module reg8(input clk, rst_n, input [7:0] d, output reg [7:0] q);
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 8'b0;
        else        q <= d;
endmodule
```

```
// 4 位双向移位寄存器（模拟 74HC194）
module shift_reg4(
    input  clk, rst_n, sin_r, sin_l,
    input  [1:0] s,            // 00=保持，01=右移，10=左移，11=置数
    input  [3:0] d,            // 并行数据输入
    output reg [3:0] q
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) q <= 4'b0;
        else case (s)
            2'b00: q <= q;
            2'b01: q <= {sin_r, q[3:1]};  // 右移：sin_r 从高位移入
            2'b10: q <= {q[2:0], sin_l};  // 左移：sin_l 从低位移入
            2'b11: q <= d;                // 并行置数
        endcase
    end
endmodule
```

5.7.2 同步计数器

```
// 4 位同步二进制加法计数器（模拟 74HC163，同步清零）
module cnt4_163(
    input  clk, clr_n, load_n, enp, ent,
    input  [3:0] d,
    output reg [3:0] q,
    output wire rco
);
    assign rco = (q == 4'b1111) & ent;  // 进位输出
    always @(posedge clk) begin
        if (!clr_n)        q <= 4'b0;          // 同步清零（163）
        else if (!load_n)  q <= d;              // 同步置数
        else if (enp & ent) q <= q + 1'b1;     // 计数
        // else: 保持（q 不变）
    end
endmodule

// 模 N 计数器（通用，置零法）
module cnt_n #(parameter N=10) (
    input  clk, rst_n,
    output reg [$clog2(N)-1:0] q,
    output wire carry
);
    assign carry = (q == N-1);
    always @(posedge clk or negedge rst_n)
        if (!rst_n || carry) q <= 0;  // 复位或到达 N-1 时清零
        else                 q <= q + 1;
endmodule
```

5.7.3 序列检测器（状态机）

```
// 序列检测器：检测输入串行序列中是否出现 101（Mealy 型）
module seq_det_101(
    input  clk, rst_n, x,  // x: 串行输入
    output reg z           // z: 检测结果（出现 101 时为 1）
```

```
);
    // 状态编码（独热码）
    localparam S0=2'd0, S1=2'd1, S2=2'd2, S3=2'd3;
    reg [1:0] state;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin state <= S0; z <= 0; end
        else case (state)
            S0: begin state <= x ? S1 : S0; z <= 0; end
            S1: begin state <= x ? S1 : S2; z <= 0; end
            S2: begin
                if (x) begin state <= S3; z <= 1; end  // 检测到 101!
                else   begin state <= S0; z <= 0; end
            end
            S3: begin state <= x ? S1 : S2; z <= 0; end
            default: begin state <= S0; z <= 0; end
        endcase
    end
endmodule
```

5.7.4 Verilog 时序逻辑设计规范

设计时序逻辑电路的 Verilog 规范要点：

always 块的敏感列表只包含时钟沿和异步复位信号，不要加入其他信号，否则综合结果可能不符合预期。

时序逻辑中统一使用非阻塞赋值（<=），组合逻辑中使用阻塞赋值（=），严禁在同一 always 块中混用。

case 语句必须覆盖所有可能取值，加上 default 分支，防止综合出意外的锁存器。

状态机建议使用独热码（One-Hot Encoding）或格雷码（Gray Code），前者在 FPGA 中速度更快，后者在低功耗 ASIC 中更节能。

异步复位的推荐写法：negedge rst_n 在敏感列表中，if(!rst_n)在最高优先级；同步复位仅用 posedge clk 在敏感列表，if(!rst_n)在 clk 触发后判断。

本章小结

本章系统介绍了时序逻辑电路的基本理论和工程应用。

在基本概念方面：时序电路由组合逻辑和存储电路（触发器）组成，具有状态记忆功能。三组核心方程（驱动方程、状态方程、输出方程）完整描述时序电路行为。按时钟同步性分为同步（各触发器共用 CLK）和异步（各触发器 CLK 可不同）两类；按输出特性分为 Moore 型（输出只与状态有关）和 Mealy 型（输出与状态和输入均有关）。

在寄存器方面：基本寄存器实现 n 位并行数据的同步存储（PIPO）。移位寄存器（以 74HC194 为例）通过 S1S0 控制端实现保持、右移、左移和并行置数四种功能，是串并/并串转换的核心器件。环形计数器和扭环形计数器是移位寄存器的典型派生应用，分别有 n 个和 2n 个有效状态。

在计数器方面：异步计数器（行波进位）结构简单但速度受限；同步计数器各位同步翻转，速度快是主流。集成 74HC161/163 是功能最全的 4 位同步二进制计数器，通过~CLR（异步/同步清零）、~LOAD（同步置数）、ENP/ENT（计数使能）、RCO（进位输出）可灵活实现任意模数计数器和多片级联扩展。置零法和置数法是构成任意 N 进制计数器的标准方法，前者检测状态 N 触发清零，后者在计到 N-1 后强制置数回初始值。

在时序电路分析方面：七步标准流程：读图→驱动方程→输出方程→代入特征方程得状态方程→状态转换表→状态转换图→描述功能。关键是正确利用触发器特征方程将驱动方程转化为状态方程。

在时序电路设计方面：七步标准流程：逻辑抽象建状态图→状态化简→编码分配→建状态转换表→推导驱动/输出方程（利用激励表+卡诺图化简）→画逻辑图→检查自启动。序列检测器是状态机设计的经典范例，Mealy 型与 Moore 型各有优缺点。自启动设计确保无效状态在有限步内进入有效循环，是时序电路可靠性的重要保证。

本章习题

一、基本概念题

5-1 举例说明时序逻辑电路与组合逻辑电路在结构和功能上的根本区别，各列举三类典型电路。

5-2 Moore 型和 Mealy 型时序电路的区别是什么？各适合什么应用场景？请设计一个简单例子分别用两种方式实现'检测 X=1 持续两个时钟周期'的功能，比较状态数的差异。

5-3 解释同步计数器和异步计数器的工作原理差异，说明异步计数器的速度为何受限，并分析为何异步计数器在状态转换时可能产生竞争冒险。

二、寄存器与移位寄存器

5-4 用 74HC194 实现如下功能：（1）上电后初始状态设为 1010；（2）连续右移 3 位；（3）切换为左移模式左移 1 位；（4）最终读出 Q3Q2Q1Q0 的值。写出 S1S0 的控制序列，并逐步计算每次操作后寄存器的内容。

5-5 4 位扭环形计数器（Johnson 计数器）的初始状态为 0000，列出完整的 8 个状态序列（Q3Q2Q1Q0），并说明与环形计数器（初始状态 1000）的状态数量差异及其原因。

5-6 用移位寄存器设计一个 5 位 LFSR（线性反馈移位寄存器），反馈多项式为 x^5+x^3+1（即 Q2 和 Q4 异或后反馈到 Q0 的 D 端），初始状态 10000，计算最多能产生多少个不同状态（理论最大为 2^5-1=31 个）。

三、计数器

5-7 用 74HC163（同步清零）和最少的逻辑门，采用置零法构成七进制计数器（0~6 循环）：（1）画出连接电路图；（2）写出检测信号的逻辑表达式；（3）验证自启动性（无效状态 7=111 能否自动回到有效循环）。

5-8 用 74HC161（异步清零）和最少逻辑，采用置数法构成计数范围为 3~12（共 10 个状态）的计数器：（1）确定初始置数值；（2）确定触发置数的检测条件；（3）画出连接电路图；（4）指出计数器的模数。

5-9 两片 74HC161 级联，计数范围为 0~99（十进制两位 BCD 计数器）：（1）说明两片的连接方式；（2）分别对两片添加什么反馈逻辑实现 BCD 计数（每片各自计 0~9，低位满 10 向高位进位）；（3）画出完整电路图。

5-10 用 3 片 74HC161 级联，通过整体置零法实现模 1000（0~999）的十进制计数器，画出电路图并分析最高计数频率（设每片 74HC161 的最高计数频率为 50MHz，每个门延迟 5ns）。

四、时序电路分析

5-11 分析如下时序电路：2 个 JK 触发器（FF0、FF1），驱动方程为 J0=~Q1, K0=1, J1=Q0, K1=~Q0。无外部输入，输出 Y=Q1。（1）写出状态方程；（2）列出完整状态转换表（4 行）；（3）画出状态转换图；（4）描述逻辑功能；（5）检查自启动性。

5-12 分析含外部输入 X 的时序电路：2 个 D 触发器，驱动方程为 D0=X·~Q0+~X·Q0（即 D0=X⊕Q0），D1=Q0·Q1+Q0·X。输出 Z=Q1。（1）写出状态方程；（2）列出状态转换表（8 行：4 种现态×2 种输入）；（3）画出 Mealy 型状态转换图；（4）描述电路功能。

五、时序电路设计

5-13 设计一个同步五进制加法计数器（0→1→2→3→4→0），用 D 触发器实现：（1）确定所需触发器数量；（2）列出状态编码和转换表；（3）用卡诺图推导驱动方程；（4）验证无效状态的自启动性；（5）画出逻辑电路图。

5-14 设计一个序列检测器，输入 X 为串行信号，当检测到'1100'时输出 Z=1（Mealy 型，重叠检测）：（1）建立原始状态图（分析所有可能的前缀匹配情况）；（2）进行状态化简（若有等价状态则合并）；（3）用格雷码进行状态编码；（4）建立状态转换表；（5）用 D 触发器推导驱动方程（可用卡诺图化简，充分利用无关项）；（6）验证自启动。

5-15 用 Verilog HDL 实现一个可控计数器：输入 CLK、RST（同步复位）、UP（方向控制，1=加法，0=减法）、LOAD（置数使能）、DIN[3:0]（预置数）；输出 QOUT[3:0]（当前计数值）和 COUT（进位/借位）。要求：LOAD=1 时在 CLK 沿同步载入 DIN；RST=1 时同步清零；正常工作时根据 UP 加法或减法计数（4 位，模 16）；COUT 在加法达到 1111 或减法达到 0000 时为 1。编写模块和对应的 Testbench，测试置数、加法、减法三种模式。

5.8 时序电路的时序分析与工程设计考量

本节从工程实践角度，深化对时序逻辑电路时序特性的理解，介绍关键时序参数的计算方法以及在实际数字系统设计中必须注意的工程问题。

5.8.1 同步时序电路的时序约束

同步时序电路的最高工作时钟频率（fmax）由关键路径（Critical Path）决定。关键路径是从某个触发器输出到下一个触发器输入之间，传播延迟最大的信号路径。设关键路径上各延迟为：

触发器时钟到输出延迟 tcq（从 CLK 有效沿到 Q 端稳定的时间）、组合逻辑路径延迟 tlogic（从触发器 Q 端经组合电路到下一触发器 D 端的延迟）、下一触发器建立时间 tsu（D 端需提前于 CLK 稳定的时间）。则时序约束为：

其中 TCLK = 1/fmax 是时钟周期。因此最高工作频率：

以 74HC161 为例，典型参数为：tcq ≈ 18ns，tsu ≈ 20ns，若计数器直接级联（tlogic 主要是 RCO 到 ENT 的连线延迟，约 2ns），则单片 fmax ≈ 1/(18+2+20) = 25MHz。若将两片级联，低位片的 RCO 还需传到高位片的 ENP/ENT，增加约 2ns 额外延迟，fmax 约 24MHz，基本不变，级联不显著降低最高频率（只要 ENP/ENT 信号来自组合逻辑的传播延迟不成为瓶颈）。

保持时间（Hold Time）约束也必须满足：从 CLK 有效沿到 D 端信号改变的最短时间不能小于触发器的保持时间 th。对同步电路而言，若所有数据路径延迟大于 tcq + th（包含时钟偏斜），则保持时间约束自动满足。时钟偏斜（Skew）过大会导致保持时间违例，是高速同步电路设计的主要挑战之一。

5.8.2 时钟域与亚稳态

在实际系统中，往往存在多个不同频率的时钟（多时钟域系统，如 CPU 核心时钟 500MHz、USB 接口时钟 48MHz、串行通信时钟 9600Hz），当信号从一个时钟域跨越到另一个时钟域时，就会出现时钟域交叉（CDC）问题。

CDC 的危险在于：来自 A 时钟域的信号，被 B 时钟域的触发器采样时，若采样时刻恰好落在 A 域信号的变化窗口内（违反建立/保持时间），触发器可能进入亚稳态，输出既不是 0 也不是 1 的中间电压，经过一段时间（resolution time）后随机稳定到 0 或 1，导致 B 域读到错误值。

标准解决方案——双触发器同步器：对单比特信号，将其先通过第一个使用 B 域时钟的 D 触发器，再通过第二个 D 触发器，用两级延迟给第一个触发器足够时间解决亚稳态。第二个触发器读到的信号亚稳态概率极低（约低于 10^-12），满足工程可靠性要求。

对多比特总线的 CDC，不能简单地每位分别用双触发器同步，因为各位延迟不同会导致读到的数据组合不一致（Metastability Corruptoin）。必须使用握手协议（Request/Acknowledge 握

手）、格雷码编码（相邻值只差 1 位，即便偶尔错误也在允许误差范围内）或异步 FIFO（以异步握手协议传递数据指针）来保证多比特数据的正确传输。

5.8.3 时序电路的功耗分析

数字时序电路的功耗主要分为两类：静态功耗（Static Power）和动态功耗（Dynamic Power）。

在现代 CMOS 工艺中，动态功耗是主要成分。每个触发器和逻辑门的节点电容在每次翻转时都要充放电，产生动态功耗：

其中 α 是信号翻转活动因子（0~1，表示每个时钟周期平均翻转的概率），C 是节点等效电容，VDD 是电源电压，fCLK 是时钟频率。

降低功耗的主要手段：（1）降低电源电压 VDD——功耗与 VDD^2 成正比，效果最显著但受性能限制；（2）降低时钟频率——不需要时降低频率或关闭时钟（Clock Gating），减少无效翻转；（3）时钟门控（Clock Gating）——对不需要更新的寄存器屏蔽时钟，将其α降为 0，这是 ASIC 设计中最常用的功耗优化手段；（4）多电压域（Multi-VDD）——高性能模块用高电压，低功耗模块用低电压；（5）减少信号翻转次数——优化逻辑结构避免不必要的毛刺和中间状态。

以一个 8 位计数器为例，若 VDD=1.8V，CTotal=50fF，fCLK=100MHz，各位平均翻转率（二进制计数时最低位翻转率 0.5，次低位 0.25，依此类推，平均约 0.25），则动态功耗约为 $P = 0.25 \times 50\times10^{-15} \times (1.8)^2 \times 100\times10^6 \approx 4\mu W$。虽然单个计数器功耗极小，但一个现代 SoC 中有数亿个触发器，总功耗就相当可观。

5.9 有限状态机的工程设计实践

有限状态机（FSM，Finite State Machine）是时序逻辑设计的核心抽象工具。几乎所有控制逻辑——从简单的交通灯控制器到复杂的网络协议处理器——都可以建模为有限状态机。本节从工程实践角度深化状态机设计能力。

5.9.1 状态编码方式的选择

状态编码方式直接影响状态机的面积、速度和功耗。主要编码方式有：

编码方式	特点	适用场合	示例（4 状态）
二进制编码（顺序编码）	触发器数量最少（log2N 个），组合逻辑复杂	节省面积，速度要求不极端	00,01,10,11
格雷码	相邻状态只差 1 位，减少毛刺和功耗	低功耗，输出稳定性要求高	00,01,11,10
独热码（One-Hot）	每状态 1 个触发器（N 个触发器），组合逻辑最简，速度最快	FPGA 设计（LUT 资源丰富），速度优先	0001,0010,0100,1000
约翰逊码	n 个触发器实现 2n 状态，	顺序脉冲发生，避	0000,1000,1100,1110,1111...

	相邻状态 1 位差	免毛刺	

FPGA 设计中，独热码通常是首选：FPGA 的 LUT 资源丰富，可以高效实现独热码的次态逻辑；独热码的每个状态对应唯一触发器，次态逻辑往往只需检查 1 个或少数几个触发器的值，组合逻辑路径短，提升 fmax；Xilinx 和 Intel 的综合工具都提供自动独热编码属性（FSM_ENCODING=ONE_HOT），使设计者无需手动指定编码。

ASIC 设计中，由于触发器面积昂贵，二进制编码更常用，通过卡诺图仔细优化组合逻辑，在面积和速度间取得平衡。

5.9.2 状态机的完整 Verilog 实现范式

一个结构清晰、易于综合和验证的状态机 Verilog 模板应包含三段：第一段描述状态寄存器（时序逻辑）；第二段描述次态逻辑（组合逻辑）；第三段描述输出逻辑。以下给出通用模板：

```
// 三段式状态机模板（推荐工程写法）
module fsm_template (
    input  wire clk, rst_n, x,    // 时钟、复位、输入
    output reg  z                  // 输出
);
    // ── 第一段：状态编码（localparam） ──
    localparam [1:0]
        S0 = 2'b00,  // 初始状态
        S1 = 2'b01,
        S2 = 2'b10,
        S3 = 2'b11;

    reg [1:0] cur_state, nxt_state;

    // ── 第二段：状态寄存器（只有时序逻辑，不含任何功能判断）──
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) cur_state <= S0;
        else        cur_state <= nxt_state;
    end

    // ── 第三段：次态逻辑（纯组合逻辑，敏感列表用@(*)） ──
    always @(*) begin
        nxt_state = cur_state;  // 默认保持，防止锁存器
        case (cur_state)
            S0: nxt_state = x ? S1 : S0;
            S1: nxt_state = x ? S1 : S2;
            S2: nxt_state = x ? S3 : S0;
            S3: nxt_state = x ? S1 : S2;
            default: nxt_state = S0;  // 处理无效状态
        endcase
    end

    // ── 第四段：输出逻辑（Mealy 型包含 x，Moore 型只含 cur_state）──
```

```
    always @(*) begin
        z = 1'b0;  // 默认输出
        if (cur_state == S2 && x == 1'b1)
            z = 1'b1;  // 检测到 101 序列
    end
endmodule
```

三段式状态机的优点：结构清晰，时序逻辑与组合逻辑完全分离，综合工具能正确推断触发器和组合逻辑；次态逻辑的 default 分支处理无效状态，保证自启动；输出逻辑独立处理，便于在 Moore 型和 Mealy 型之间切换；整体可读性和可维护性强，是工业界推荐的 FSM 写法规范。

5.9.3 状态机的验证与调试

状态机的正确性验证是设计流程中不可省略的关键步骤。在仿真阶段，Testbench 应当覆盖以下几类测试场景：

- 正常功能测试：按设计要求的输入序列，逐步验证状态转换和输出是否与设计文档（状态转换表）完全一致。
- 边界条件测试：在状态机处于每个状态时，测试所有可能的输入值，确保次态和输出在所有情况下正确。
- 复位功能测试：验证异步复位（或同步复位）能否在任意状态下正确将状态机回到初始状态 S0。
- 无效状态自启动测试：强制将状态机设置到每个无效（不应出现）状态，验证经过有限步骤后能否自动进入有效状态循环。

在 FPGA 实现后，可以利用片上调试工具（如 Xilinx 的 ILA，Intel 的 SignalTap）对状态机的内部状态进行实时采样和显示，这对于发现仅在真实硬件环境中才会出现的时序问题（如 CDC 引起的亚稳态、时钟抖动导致的建立时间违例）非常有效。

5.10 综合设计案例

本节通过两个综合性设计案例，将本章所学的移位寄存器、计数器和状态机知识整合应用，展示从需求分析到完整实现的全流程。

5.10.1 案例一：数字秒表（60 进制计数器）

设计要求：实现一个 0~59 的十进制两位数秒计数器，能够以 1Hz 时钟（周期 1 秒）驱动，显示 0~59 秒循环计数，并在满 60 秒时产生一个进位脉冲。

设计方案：用两片 74HC161（或 74HC163）构成两级 BCD 计数器，分别对应个位（0~9）和十位（0~5），共同实现 60 进制计数。

个位计数器（片 1）：计数 0~9（十进制），利用置零法，当 Q3·Q0=1（状态 10=1010，即十进制 10）时清零。实际上更简洁的方案是：检测进位时刻用 RCO，当个位计到 9

（1001）时 RCO=1，但 RCO 在 1111 时才为 1，所以个位计数器需要在计到 10 时置零。检测条件：Q3·Q1=1（10=1010，Q3=1 且 Q1=1）。

十位计数器（片 2）：计数 0~5（六进制），利用置零法，当十位计数器的 Q2·Q1=1（状态 6=0110，Q2=1 且 Q1=1）时清零。

进位协调：个位满 10（清零）时，需向十位产生一个进位脉冲。用个位计数器清零信号的下降沿（或单独检测个位=9 且下一 CLK 时的状态）触发十位计数。最终整体输出用两组七段译码器驱动两位数码管显示。

```
// 60 进制计数器（BCD 格式）
module cnt60 (
    input  wire clk,        // 1Hz 时钟
    input  wire rst_n,      // 异步复位
    output reg  [3:0] ones, // 个位 BCD
    output reg  [3:0] tens, // 十位 BCD
    output wire carry       // 60s 进位脉冲
);
    // 个位：0~9 循环
    wire ones_carry = (ones == 4'd9);
    always @(posedge clk or negedge rst_n)
        if (!rst_n)        ones <= 0;
        else if (ones_carry) ones <= 0;  // 满 10 清零
        else                 ones <= ones + 1;

    // 十位：个位进位时加 1，到 5 后清零
    wire tens_carry = (tens == 4'd5) & ones_carry;
    always @(posedge clk or negedge rst_n)
        if (!rst_n)         tens <= 0;
        else if (tens_carry) tens <= 0;  // 满 6 清零（十位）
        else if (ones_carry) tens <= tens + 1;

    assign carry = tens_carry;  // 60 秒时输出进位
endmodule
```

此设计只需约 40 行 Verilog 代码即可实现完整的 60 进制 BCD 计数器，综合后将产生 2 个 4 位计数器寄存器（8 个 D 触发器）加少量组合逻辑。通过级联多个此类模块（分、秒、小时），可以构成完整的数字时钟计数系统，是数字系统综合实验的经典题目之一。

5.10.2 案例二：串行数据接收状态机（UART 起始位检测）

UART（通用异步收发传输器）是嵌入式系统中最常用的串行通信接口。在 UART 协议中，数据帧以一个低电平的起始位（Start Bit）开始，然后是 8 位数据位，最后以高电平的停止位（Stop Bit）结束。接收端需要检测起始位的到来，以正确同步数据采样。

起始位检测状态机（简化版）：UART 接收端使用远高于波特率的采样时钟（通常 16 倍采样），当检测到 RXD 信号下降沿时，启动计数，在起始位的中点（8 个采样周期后）确认是真正的起始位（而不是噪声干扰），然后开始按波特率周期采样 8 个数据位。

状态机状态定义：IDLE-空闲等待（RXD=1）；START_DET-检测到下降沿，等待确认；SAMPLE-逐位采样数据；STOP-等待停止位。

```
// UART 起始位检测与数据采样状态机（简化）
module uart_rx (
    input  wire       clk16x,    // 波特率 16 倍的采样时钟
    input  wire       rst_n,
    input  wire       rxd,        // 串行输入
    output reg  [7:0] data,       // 接收到的 8 位数据
    output reg        data_valid // 数据有效脉冲
);
    localparam IDLE=2'd0, START=2'd1, DATA=2'd2, STOP=2'd3;
    reg [1:0]  state;
    reg [3:0]  cnt;     // 16 倍过采样计数
    reg [2:0]  bit_cnt;// 已接收数据位数
    reg [7:0]  shift;  // 移位寄存器

    always @(posedge clk16x or negedge rst_n) begin
        if (!rst_n) begin
            state <= IDLE; cnt <= 0; bit_cnt <= 0; data_valid <= 0;
        end else begin
            data_valid <= 0;  // 默认无效
            case (state)
            IDLE: if (!rxd) begin state <= START; cnt <= 0; end
            START: begin
                cnt <= cnt + 1;
                if (cnt == 4'd7) begin  // 起始位中点
                    if (!rxd) begin state <= DATA; cnt <= 0; bit_cnt <= 0; end
                    else      state <= IDLE;  // 噪声，返回等待
                end
            end
            DATA: begin
                cnt <= cnt + 1;
                if (cnt == 4'd15) begin  // 每 16 个采样取一次
                    shift <= {rxd, shift[7:1]};  // 低位先入
                    bit_cnt <= bit_cnt + 1;
                    if (bit_cnt == 3'd7) state <= STOP;
                end
            end
            STOP: begin
                cnt <= cnt + 1;
                if (cnt == 4'd15) begin
                    if (rxd) begin data <= shift; data_valid <= 1; end
                    state <= IDLE;
                end
            end
            endcase
        end
    end
endmodule
```

该状态机展示了在实际通信接口设计中，时序逻辑（触发器寄存器）、计数器（cnt、bit_cnt）、移位寄存器（shift）和有限状态机（state）这四类时序电路元素如何协同工作，共

同实现一个完整的功能协议处理逻辑。这也是本章所有内容在一个实际工程案例中的综合体现。

5.11 习题详解精选

本节对本章重点习题给出完整的解题思路和关键步骤，帮助读者掌握时序电路分析与设计的标准方法。

5.11.1 计数器分析典型例题

例题：分析两片 74HC161 级联实现模 100 计数器的电路

设计要求：两片 74HC161，低位片计个位（0~9），高位片计十位（0~9），实现 0~99 的 BCD 格式十进制计数，低位片满 10 产生进位给高位片。

分析：个位片（片 1）使用置零法，检测状态 10（Q3·Q1=1）触发~CLR 清零，同时这个清零信号通过非门转为正脉冲，加上一个脉冲延展（或直接用个位片的 RCO）驱动十位片的 ENP/ENT。但 74HC161 的 RCO 在计到 1111 时才为 1，不适合十进制计数器的级联。

正确方案——两片独立 BCD 计数，行波进位：个位片独立实现 0~9（置零法），当个位清零时（下降沿）产生一个时钟给十位片。但为了保持同步，更好的方案是：个位片到达 9（1001）时，RCO 概念上应产生进位，但标准 RCO 在 1111 才有效。因此改用 ENT 信号控制：当个位片的 Q3·Q0=1（状态 9=1001）且下一 CLK 会清零时，使十位片的 ENT=1，让十位片在同一 CLK 沿加 1。

这种方案的 Verilog 实现即前面给出的 cnt60 模块的个位/十位分离写法，用行为级描述比门级连线更清晰，且综合结果同样高效。

5.11.2 状态机设计典型例题精解

例题：设计同步模 5 计数器（用 JK 触发器，带自启动）

第一步：确定有效状态和编码（3 位：Q2Q1Q0）

5 个有效状态：S0=000, S1=001, S2=010, S3=011, S4=100，无效状态：101,110,111。

第二步：建立状态转换表（含无效状态的无关项处理）

Q2Q1Q0（现态）	Q2+Q1+Q0+（次态）	J2K2	J1K1	J0K0
000（S0）	001（S1）	0x	0x	1x
001（S1）	010（S2）	0x	1x	x1
010（S2）	011（S3）	0x	x0	1x
011（S3）	100（S4）	1x	x1	x1
100（S4）	000（S0）	x1	0x	0x
101（无效）	（→000）	xx	xx	xx
110（无效）	（→000）	xx	xx	xx
111（无效）	（→000）	xx	xx	xx

第三步：用卡诺图化简各激励端（充分利用无关项×）

第四步：验证自启动。代入无效状态 101：J2=0·1=0，K2=1→Q2+=0；J1=0·1=0，K1=1→Q1+=0；J0=0，K0=1→Q0+=0，次态 000，进入有效循环。状态 110：J2=1·0=0，K2=1→Q2+=0；J1=0·0=0，K1=0→Q1+=1；J0=0，K0=1→Q0+=0，次态 010，进入有效循环。状态 111：J2=1·1=1，K2=1→Q2+=0；J1=0·1=0，K1=1→Q1+=0；J0=0，K0=1→Q0+=0，次态 000，进入有效循环。所有无效状态均能在 1 步内进入有效循环，自启动能力强。

5.11.3 移位寄存器应用典型例题

例题：用 74HC194 实现 4 位并串转换（PISO，并行输入串行输出）

要求：在一个 CLK 周期内将 D0~D3 并行载入，然后在接下来 4 个 CLK 周期将数据从 Q3 端串行输出（高位先出）。

控制序列设计：第 1 周期，S1S0=11（并行置数），将 D0~D3 载入；第 2~5 周期，S1S0=01（右移），每个 CLK 将 Q3 的值输出，同时数据向右移位；Sin_R 接 0（移位时从高位补 0）；4 个 CLK 后数据全部串行输出完毕。

关键分析：右移操作时，最高位（Q3）先输出，随后 Q2、Q1、Q0 依次出现在 Q3 端。若要低位先出，改用左移（S1S0=10），从 Q0 端输出，Sin_L 接 0。

时序验证（设初始数据 D3D2D1D0=1011）：

CLK 沿	S1S0	操作	Q3Q2Q1Q0	Q3 输出
第 1 沿	11	并行置数	1011	—
第 2 沿	01	右移（高位先出）	0101	1（D3）
第 3 沿	01	右移	0010	0（D2）
第 4 沿	01	右移	0001	1（D1）
第 5 沿	01	右移	0000	1（D0）

串行输出序列（Q3 端）：1,0,1,1——与 D3D2D1D0=1011 完全一致（高位先出顺序），验证设计正确。

本章重要公式与参数汇总

以下汇总本章核心公式和典型器件参数，供学习参考。

核心公式

常用器件参数速查

器件	功能	关键控制端	典型 fmax	备注
74HC161	4 位同步二进制计数器（异步清零）	~CLR,~LOAD,ENP,ENT,RCO	50MHz@5V	~CLR 异步，~LOAD 同步
74HC163	4 位同步二进制计数器（同步清零）	CLR,~LOAD,ENP,ENT,RCO	50MHz@5V	CLR 同步，其余同 161
74HC194	4 位双向移位寄存	S1S0,Sin_R,Sin_L,~CLR	50MHz@5V	00 保持/01 右移/10

	器			左移/11 置数
74HC374	8 位上升沿 D 触发器（三态输出）	CLK,~OE	80MHz@5V	寄存器/总线接口标准芯片
74HC595	8 位串入并出移位寄存器（带锁存）	SH_CP,ST_CP,~OE	25MHz	IO 扩展首选，3 线驱动 8 位输出

5.12 FPGA 中时序逻辑电路的实现

在 FPGA（现场可编程门阵列）上实现时序逻辑电路，是现代数字工程师的核心技能之一。FPGA 提供了丰富的触发器资源和灵活的互连结构，使得从简单的移位寄存器到复杂的状态机都可以高效实现。本节介绍时序逻辑在 FPGA 上实现的关键技术要点。

5.12.1 FPGA 触发器资源特性

以 Xilinx 7 系列 FPGA 为例，每个基本逻辑单元（Slice）包含 4 个查找表（LUT）和 8 个触发器（FF）。每个触发器具有以下可配置特性：

- 时钟边沿：可配置为上升沿或下降沿触发（通过 IS_C_INVERTED 属性）。
- 时钟使能（CE）：每个触发器有独立的时钟使能端，CE=0 时无论时钟如何触发，状态保持不变。这比在时钟线上加门控更安全（避免毛刺），是 FPGA 中实现条件更新的标准方法。
- 置位/复位（SR）：可配置为同步置位（SS）、同步复位（SR）或异步置位复位。同一 Slice 内的触发器必须使用相同极性的 SR 信号，但可以是置位或复位。
- 初始值（INIT）：可以为每个触发器配置上电初始值（0 或 1），不依赖复位信号，在配置（上电加载比特流）时自动初始化，保证确定的初始状态。

FPGA 寄存器的高效使用技巧：尽量使用时钟使能（CE）而非门控时钟（Gated Clock）；合理利用 SR 端口，让同一 Slice 的触发器共享复位逻辑，减少布线资源；避免使用锁存器（Latch），FPGA 的综合工具会发出警告，且锁存器会占用 LUT 而非 FF 资源，效率低下。

5.12.2 时序收敛与约束文件

FPGA 设计中，通过 XDC（Xilinx Design Constraints）约束文件告知工具时序要求，工具据此优化布局布线，使设计满足时序收敛（Timing Closure）。基本约束写法：

```
# 创建 25MHz 主时钟约束（时钟周期 40ns）
create_clock -period 40.000 -name sys_clk [get_ports clk]

# 输入延迟约束（相对于时钟的输入数据延迟）
set_input_delay -clock sys_clk -max 5.0 [get_ports {din[*]}]
set_input_delay -clock sys_clk -min 1.0 [get_ports {din[*]}]

# 输出延迟约束
set_output_delay -clock sys_clk -max 3.0 [get_ports {dout[*]}]

# 多周期路径（某条路径允许用 2 个时钟周期）
```

```
set_multicycle_path 2 -setup -from [get_cells reg_a] -to [get_cells reg_b]

# 伪路径（不需要时序分析的路径，如 CDC 路径）
set_false_path -from [get_clocks clk_a] -to [get_clocks clk_b]
```

时序收敛的一般方法：首先运行综合和实现，查看时序报告中的负裕量（Negative Slack）路径；对关键路径，可通过优化逻辑结构（减少组合逻辑层级）、添加流水线寄存器（Pipeline Register）或使用更快的 FPGA 资源来改善；对跨时钟域路径，添加 False Path 约束告知工具该路径不需要时序检查（已由异步 FIFO 或同步器处理），避免误报时序违例。

5.12.3 综合属性与时序优化技术

在 Verilog 代码中可以通过综合属性（Synthesis Attributes）指导工具进行特定的优化：

```
// 强制状态机编码为独热码
(* fsm_encoding = "one_hot" *)
reg [3:0] state;

// 禁止寄存器被优化掉（如测试用的采样寄存器）
(* keep = "true" *)
reg debug_reg;

// 标记为流水线寄存器，工具会在此处优化时序
(* shreg_extract = "no" *)  // 禁止被自动识别为移位寄存器优化
reg pipeline_stage;

// 约束寄存器放置在特定区域（通过 XDC 配合）
// 在 XDC 中: create_pblock pb1; add_cells_to_pblock pb1 [get_cells u_counter]
```

流水线（Pipeline）是解决时序违例最常用的技术：将一个组合逻辑延迟过大的路径从中间切断，插入一级寄存器，将原来一个时钟周期完成的操作分为两个时钟周期完成。代价是增加了一个时钟周期的延迟（Latency），但吞吐率（Throughput）不变或提高。例如，一个 16 位乘法器路径延迟 30ns，100MHz 时钟（10ns）无法满足；插入两级流水线后，每级约 15ns，100MHz 时钟可以满足，且每个时钟周期仍能输出一个乘法结果（吞吐率不变），只是结果滞后 2 个时钟周期。

5.13 时序逻辑电路测试与调试方法

数字系统设计中，测试与调试的重要性不亚于设计本身。本节介绍时序电路从仿真验证到硬件调试的全流程方法。

5.13.1 仿真验证策略

时序电路的仿真验证应当分层次进行：单元验证（Unit Test）、集成验证（Integration Test）和系统验证（System Test）。

单元仿真（Testbench 编写要点）

一个高质量的时序电路 Testbench 应当包含：时钟生成（用 initial + forever 实现精确周期的时钟）；复位序列（上电后先复位若干周期，再释放复位，进入正常工作）；功能激励（按照状态转换表，依次施加各种输入组合，覆盖所有状态和转换）；输出检查（用$display 输出或用 if/assert 自动比较期望值与实际值）；边界和压力测试（连续输入、快速切换、极端状态等）。

```
// 时序电路 Testbench 标准模板
module tb_cnt60;
    reg  clk, rst_n;
    wire [3:0] ones, tens;
    wire carry;

    cnt60 DUT(.clk(clk),.rst_n(rst_n),.ones(ones),.tens(tens),.carry(carry));

    // 时钟生成（10ns 周期，即 100MHz，实际 60 进制计数器用 1Hz）
    initial clk = 0;
    always #5 clk = ~clk;  // 每 5ns 翻转，周期 10ns

    // 复位和测试序列
    initial begin
        rst_n = 0; #12;    // 复位保持 12ns
        rst_n = 1;          // 释放复位，开始计数
        #700;               // 运行 70 个时钟周期（> 60 个状态）

        // 验证 carry 信号（应在第 60 个 CLK 后出现）
        if (carry) $display('PASS: carry detected at t=%0t', $time);
        else        $display('FAIL: carry not detected');
        $finish;
    end

    // 实时监控输出
    always @(posedge clk)
        $display('t=%0t: tens=%0d ones=%0d carry=%b', $time, tens, ones,
carry);
endmodule
```

5.13.2 常见时序电路 bug 及其排查

时序逻辑设计中最常见的错误类型及其排查方法：

错误类型	症状	常见原因	排查方法
锁存器推断	综合报告出现 Latch，仿真结果异常	always 块中 if/case 未覆盖所有分支，组合逻辑变量未在所有路径赋值	检查所有 always @(*)块，确保 default 分支存在，所有输出变量在所有分支都被赋值
时序违例	仿真正确但 FPGA 上偶发错误	组合逻辑延迟超过时钟周期，时序分析报告 Negative Slack	查看时序报告的关键路径，添加流水线或优化逻辑
复位问题	上电后状态不确定，行为不可预测	缺少复位逻辑，或异步复位信号未同步释放	确保所有触发器有复位端，异步复位推荐使用同步释放
亚稳态	随机性错误，特别在不同温度/频率下	跨时钟域未做同步处理	所有 CDC 信号加双触发器同步器或异步 FIFO

阻塞/非阻塞混用	仿真与综合行为不一致	时序逻辑中误用阻塞赋值=	时序 always 用<=，组合 always 用=，严格分离

5.13.3 片上调试（In-System Debugging）

当仿真通过但硬件行为异常时，需要对 FPGA 内部信号进行实时采样和分析。主要工具有：

- Xilinx ILA（Integrated Logic Analyzer）：在设计中插入 ILA IP 核，选择需要观测的信号，设置触发条件（如某状态机进入特定状态），运行后在 Vivado 中实时查看波形。是 FPGA 调试的首选工具。
- Intel SignalTap：功能类似于 Xilinx ILA，适用于 Intel/Altera FPGA。通过 Quartus 配置采样深度和触发条件，在 FPGA 运行时采集内部信号。
- JTAG 调试接口：通过标准 JTAG 接口读写 FPGA 内部寄存器的当前值，适合静态状态检查（无法捕获动态波形）。

调试方法论：先确认时钟和复位信号正常（用 ILA 采样时钟频率和复位波形）；然后验证状态机能进入期望状态（观察 state 信号序列）；再检查各控制信号的时序关系（ILA 的触发和采样机制精确到一个时钟周期）；最后通过分频缩短时钟来放慢电路运行速度，方便观察逻辑行为。

$$T_0 = 1 \quad \text{（最低位每周期翻转）}$$

$$T_1 = Q_0 \quad \text{（Q0=1 时 Q1 翻转）}$$

$$T_2 = Q_0 \cdot Q_1 \quad \text{（Q0=Q1=1 时 Q2 翻转）}$$

$$T_3 = Q_0 \cdot Q_1 \cdot Q_2 \quad \text{（低三位全 1 时 Q3 翻转）}$$

$$T_0 = 1, \quad T_1 = Q_0, \quad T_2 = Q_0 Q_1, \quad T_3 = Q_0 Q_1 Q_2$$

$$Q_0^{n+1} = 1 \oplus Q_0^n = \overline{Q_0^n}$$

$$Q_1^{n+1} = Q_0^n \oplus Q_1^n$$

$$Q_2^{n+1} = Q_0^n Q_1^n \oplus Q_2^n$$

$$Q_3^{n+1} = Q_0^n Q_1^n Q_2^n \oplus Q_3^n$$

$$Q_0^{n+1} = \overline{Q_2^n}, \quad Q_1^{n+1} = Q_0^n, \quad Q_2^{n+1} = Q_1^n$$

$$D_1 = Q_0 \overline{Q_1}, \quad D_0 = \overline{Q_0}\,\overline{Q_1}$$

$$D_1 = X \cdot \overline{Q_1} + X \cdot Q_0 + \overline{X} \cdot Q_1 \cdot \overline{Q_0}$$

$$D_0 = X \cdot \overline{Q_0}$$

$$Z = X \cdot Q_1 \cdot Q_0 \quad \text{（Mealy 型输出）}$$

$$J_2 = Q_1 Q_0, \ K_2 = Q_0 + Q_1; \quad J_1 = \overline{Q_2} Q_0, \ K_1 = Q_0; \quad J_0 = \overline{Q_2}, \ K_0 = 1$$

$$t_{cq} + t_{logic} + t_{su} \le T_{CLK}$$

$$f_{max} = \frac{1}{t_{cq} + t_{logic,\,max} + t_{su}}$$

$$P_{dynamic} = \alpha \cdot C \cdot V_{DD}^2 \cdot f_{CLK}$$

$$J_2 = Q_1 Q_0, \quad K_2 = 1$$

$$J_1 = \overline{Q_2} Q_0, \quad K_1 = Q_0$$

$$J_0 = \overline{Q_2}, \quad K_0 = 1$$

同步计数器翻转条件：$T_i = Q_0 \cdot Q_1 \cdots Q_{i-1}$

计数器模数与位数关系：$M = 2^n$（n 位二进制计数器）

时序约束：$f_{max} = \dfrac{1}{t_{cq} + t_{logic,\,max} + t_{su}}$

动态功耗：$P = \alpha \cdot C \cdot V_{DD}^2 \cdot f_{CLK}$

LFSR 最大序列长度：$L_{max} = 2^n - 1$（n 位寄存器，适当反馈）